

Incident Management Case Studies

Lessons Learned from Major Incidents — 2025 Collection | SRE Team

1. Case Study: The Black Friday Database Meltdown (INC-2025-089)

On November 28, 2025 (Black Friday), TechViet experienced a severe database performance degradation that affected the ChatAssist platform for approximately 4 hours and 23 minutes, resulting in degraded service for over 150,000 active users and an estimated revenue impact of VND 2.3 billion in lost enterprise customer SLA credits. This case study documents the incident timeline, root cause analysis, and lessons learned.

1.1 Incident Timeline

08:00 UTC (15:00 Vietnam time) — The ChatAssist platform begins experiencing elevated API response times as Black Friday promotional campaigns drive a 340% increase in chat volume compared to the daily average. The Datadog monitoring dashboard shows p99 latency climbing from the normal 145ms to 400ms over a 30-minute period. The auto-scaling system successfully scales the application tier from 8 pods to 16 pods to handle the increased load. At this point, all SLOs are still within acceptable ranges.

09:15 UTC — The primary PostgreSQL database server (db-prod-01) begins showing signs of stress. The number of active connections increases from the normal 120 to 380 (approaching the max_connections limit of 400). CPU utilization on the database server reaches 92%. The database monitoring alert fires at the WARNING level, but the on-call engineer (Pham Thanh Dat, L3 Backend Engineer) acknowledges the alert and notes that the metrics are elevated but not yet critical. He begins investigating the slow query log and finds that a recently deployed analytics query — introduced 3 days earlier in PR #4521 by the Analytics team — is performing full table scans on the messages table (450 million rows) without using the created_at index. The query was intended to run as a background job during off-peak hours, but a configuration error caused it to run every 5 minutes instead of daily at 02:00 UTC.

09:45 UTC — The database connection pool is exhausted. New connection attempts from the application pods are rejected, causing HTTP 503 errors for approximately 30% of API requests. The Conversation Service's health check starts failing, causing the load balancer to remove healthy pods from the rotation (a cascading failure caused by the health check depending on the database connection). PagerDuty escalates to P1, and the Incident Commander (Le Van Huy, Engineering Manager) declares a major incident and opens the #incident-089 Slack channel.

10:00 UTC — The incident response team assembles in the war room (Zoom bridge). The IC assigns roles: Communications Lead (Nguyen Thi Lan, Product Manager) to handle customer communications, Database SME (Nguyen Van Binh, DBA) to address the database issues, and Application SME (Tran Quoc Hung, L5 Staff Engineer) to investigate the application-level failures. The first mitigation action is to kill the runaway analytics query, but this requires identifying the specific backend PID of the query, which Binh locates using pg_stat_activity. He terminates the query and the 4 scheduled instances waiting to execute, then temporarily disables the analytics job's cron schedule in the Airflow configuration.

10:15 UTC — Database connections begin to recover, dropping from 400 (max) to 250 within 5 minutes as the terminated queries release their connections. However, the application pods that were removed from the load balancer rotation have not yet recovered because their health checks require 3 consecutive successful checks at 10-second intervals (30 seconds total). During this recovery window, the remaining pods are overwhelmed, maintaining the 503 error rate at approximately 15%.

10:25 UTC —

All application pods are back in the load balancer rotation. The error rate drops to near zero, but API latency remains elevated at p99 = 800ms (compared to the normal 145ms) because the database connection pool is still recovering and the Redis cache was partially invalidated during the incident (cache entries that expired during the downtime were not refreshed, causing additional database load). The team decides to manually warm the cache by replaying the most common query patterns from the access log. 11:30 UTC — Cache warming is complete. API latency returns to normal levels (p99 = 160ms). The IC declares the incident mitigated but keeps the war room open for monitoring. 12:23 UTC — After 1 hour of stable operation with all metrics within normal ranges, the IC formally closes the incident. Total customer-facing impact: 4 hours 23 minutes of degraded service (elevated latency), with approximately 38 minutes of severe degradation (503 errors for 15-30% of requests).

1.2 Root Cause Analysis

The root cause was a combination of three factors that individually would not have caused an incident, but together created a cascading failure. Factor 1: The analytics query (introduced in PR #4521) was designed to aggregate message counts by customer for the monthly billing report. The query performed a sequential scan on the messages table because the WHERE clause included a function call on the created_at column (WHERE DATE_TRUNC('month', created_at) = '2025-11-01') that prevented the PostgreSQL query planner from using the btree index on created_at. The correct approach would have been to use a range condition (WHERE created_at >= '2025-11-01' AND created_at < '2025-12-01'), which the planner can satisfy using the index. This query was not caught during code review because: the reviewer (an Analytics team engineer) focused on the business logic correctness rather than query performance, and there is no automated mechanism to test query execution plans against production-like data volumes during the CI/CD pipeline. Factor 2: The scheduling configuration error. The analytics job was configured in Airflow with the cron expression `* / 5 * * * *` (every 5 minutes) instead of `0 2 * * *` (daily at 2:00 AM). This was a copy-paste error from a test configuration that was used during development and was not caught because the configuration file is in a separate repository (techviet/airflow-dags) that has a different code review process than the main application repositories — specifically, DAG configuration changes are auto-approved if only the schedule or parameters change, without the mandatory 2-reviewer requirement. Factor 3: The health check design flaw. The Conversation Service's health check endpoint (/healthz) performs a SELECT 1 query against the database to verify connectivity. When the database connection pool was exhausted, this health check query could not acquire a connection, causing the health check to fail. The load balancer then removed these pods from rotation, even though the pods themselves were healthy and could have served requests from the Redis cache for read operations. A better design would have been to separate the liveness probe (is the process running?) from the readiness probe (can the process serve traffic?), and to have the readiness probe degrade gracefully rather than fail completely when a single dependency is unavailable.

1.3 Remediation Actions

The postmortem review, attended by 12 engineers across 4 teams, identified the following remediation actions, all of which were completed by December 15, 2025: Action 1 (Database): Fix the analytics query to use index-friendly range conditions. Add a slow query alert that triggers when any query runs for more than 30 seconds (previously the threshold was 5 minutes). Implement query plan validation in the CI pipeline using pg_plan_guarantee, a tool that compares the query plan against production table statistics to detect potential full table scans. Owner: Nguyen Van Binh (DBA). Status: Completed. Action 2 (Scheduling): Fix the cron expression and add a validation step in the Airflow DAG deployment pipeline that requires manual approval for any schedule that runs more frequently than hourly. Add a comment in every DAG file documenting the intended schedule in human-readable format. Owner: Pham Thanh Dat (Backend Engineer). Status: Completed. Action 3 (Health Checks): Refactor all services to separate liveness and readiness probes. The liveness probe checks only that the process is running and the event

loop is not blocked. The readiness probe checks dependencies but with a configurable degradation mode: if the database is unavailable but the cache is available, the service reports 'degraded' readiness and the load balancer reduces (but does not eliminate) traffic to that pod. Owner: Tran Quoc Hung (Staff Engineer). Status: Completed. Action 4 (Process): Extend the mandatory 2-reviewer code review requirement to the Airflow DAGs repository and all configuration repositories. Add a checklist item to the PR template for database-touching changes: 'Have you verified the query execution plan against production table statistics?' Owner: Le Van Huy (Engineering Manager). Status: Completed. Action 5 (Capacity Planning): Increase the PostgreSQL max_connections from 400 to 600 and implement PgBouncer as a connection pooler to better manage connection lifecycle. Conduct quarterly capacity planning reviews that include projected traffic growth and database resource requirements. Owner: Nguyen Van Binh (DBA). Status: Completed.

2. Case Study: The Midnight Certificate Expiry (INC-2025-112)

On December 15, 2025, at 00:00 UTC (07:00 Vietnam time), the TLS certificate for `api.chatassist.techviet.vn` expired, causing all HTTPS connections to the ChatAssist API to fail with certificate validation errors. The incident lasted 2 hours and 47 minutes and affected all ChatAssist customers who integrate via the API, including 23 enterprise customers with SLA agreements. This was particularly embarrassing because TechViet's own monitoring system uses the same API endpoint, meaning that the monitoring dashboards also went dark, delaying initial detection.

2.1 Timeline and Response

The incident unfolded as follows: at 00:00 UTC, the Let's Encrypt TLS certificate for `api.chatassist.techviet.vn` expired. The certificate had been issued 90 days earlier (September 16, 2025) with the standard Let's Encrypt validity period. The automatic renewal process, managed by certbot running on the F5 load balancer, was supposed to renew the certificate 30 days before expiry (November 15, 2025). However, the renewal failed silently because the DNS validation method used by certbot required a TXT record to be created in the TechViet DNS zone (hosted on Cloudflare), and the API token used by certbot to create the DNS record had been rotated as part of a security hardening exercise on October 28, 2025, without updating the certbot configuration. The certbot log showed the renewal failure, but the log file had grown to 2 GB due to verbose logging settings and was being rotated by logrotate before the monitoring system could parse it. 00:00 UTC — Certificate expires. All API requests begin receiving TLS handshake failures. The Datadog synthetic monitoring check for the API endpoint also fails, but Datadog classifies it as a 'transport error' rather than a 'service error', and the alert routing rules only page the on-call engineer for service errors. 00:23 UTC — The first customer report arrives via the support email: 'Our integration with ChatAssist API has stopped working. We are getting SSL errors.' The support team (24/7 coverage) creates ticket SUPPORT-8842 and attempts to reproduce the issue by accessing the API documentation site (which uses a different certificate and is unaffected), incorrectly concluding that the API is operational. 00:45 UTC — Three more customer reports arrive, and the support shift lead (Nguyen Thi Huong) escalates to the on-call engineer (Le Minh Quang, L4 Platform Engineer). Quang checks the Datadog dashboard and sees that it has been showing 'No data' for the API metrics since 00:00, which he initially interprets as a monitoring issue rather than a service issue (a classic failure mode where monitoring goes blind at the same time as the service fails). He SSH's into the load balancer and discovers the expired certificate at 01:02 UTC. 01:15 UTC — Quang attempts to manually renew the certificate using certbot, but encounters the expired API token error. He does not have access to the Cloudflare dashboard (the account is managed by the Security team), so he escalates to the CISO (Le Hoang Nam) via phone. Nam logs in remotely, generates a new API token, and provides it to Quang. 01:45 UTC — The new API token is configured in certbot, and the certificate renewal process begins. The DNS validation takes 12 minutes due to TTL propagation delays. 02:00 UTC — The new certificate is issued and installed on the F5 load balancer. Quang performs a rolling restart of the load balancer's TLS

engine to pick up the new certificate without dropping existing connections. 02:12 UTC — API monitoring confirms that HTTPS connections are succeeding. All synthetic checks return to green. The on-call engineer monitors for 30 minutes to ensure stability. 02:47 UTC — Incident formally closed after confirming no residual issues. Customer communications sent to all affected enterprise customers with an initial incident summary and timeline.

2.2 Root Cause and Remediation

The root cause chain was: security hardening exercise rotated the Cloudflare API token without updating the certbot configuration, the certbot renewal failure was logged but not monitored, the monitoring system's alert routing classified the resulting TLS failures as transport errors rather than service errors, and the support team's initial triage process did not include checking the actual API endpoint (only the documentation site). This incident revealed five systemic weaknesses that the team addressed with the following remediations: Remediation 1: Implement a certificate expiry monitoring check that fires at 30 days, 14 days, 7 days, 3 days, and 1 day before expiry. This check queries the certificate directly from the TLS handshake, independent of certbot's logs. Implemented using Datadog's TLS certificate check with PagerDuty integration at the 7-day threshold. Owner: Le Minh Quang. Status: Completed December 18, 2025. Remediation 2: Create a 'credential dependency map' that documents every credential (API tokens, passwords, certificates) and the systems that depend on them. When any credential is rotated, the rotation runbook requires verifying that all dependent systems are updated. This map is stored in the Security team's wiki and reviewed quarterly. Owner: Le Hoang Nam (CISO). Status: Completed January 10, 2026. Remediation 3: Update the Datadog alert routing rules to treat transport errors (TLS failures, connection refused, DNS resolution failures) as service errors for customer-facing endpoints, triggering immediate PagerDuty escalation. Owner: DevOps team. Status: Completed December 20, 2025. Remediation 4: Migrate from Let's Encrypt with DNS validation to AWS Certificate Manager (ACM) for all externally facing endpoints, which provides automatic renewal with no manual intervention required. This migration is planned as part of Phase 4 of the cloud migration. In the interim, the Let's Encrypt configuration has been fixed with the new API token. Owner: Platform team. Status: In Progress (target: cloud migration Phase 2). Remediation 5: Update the support team's triage runbook to include actual endpoint verification (not just related endpoints) and to provide escalation paths for after-hours infrastructure issues. Owner: Support team lead. Status: Completed December 22, 2025.

3. Case Study: The Data Pipeline Silent Corruption (INC-2026-003)

In the first week of January 2026, the Finance team discovered discrepancies in the December 2025 billing reports generated by the data pipeline. The reported total message volume for three enterprise customers was approximately 15% lower than the customers' own usage tracking dashboards indicated, resulting in under-billing of approximately VND 180,000,000 across the three accounts. This incident is classified as a data integrity incident rather than a service availability incident, making it particularly insidious because the pipeline continued to operate normally — it simply produced incorrect results.

3.1 Investigation and Discovery

The investigation, led by Senior Data Engineer Vo Thi Thanh, revealed a subtle bug in the data pipeline's deduplication logic. The pipeline ingests conversation events from Kafka and writes them to the data warehouse (ClickHouse) after deduplication and enrichment. Deduplication uses the event's unique message_id as the key, and the pipeline maintains a sliding window of 1 hour of message IDs in a Redis set for fast duplicate detection. The bug was introduced on December 10, 2025, in PR #4589 by a junior data engineer who was optimizing the pipeline's memory usage. The optimization changed the Redis key format for the deduplication set from 'dedup:{topic}:{partition}:{hour}' to 'dedup:{topic}:{hour}' — removing the partition identifier. This was intended to reduce the number of Redis keys (from 48 keys per hour to 16

per hour, since each topic has 3 partitions), but it had an unintended consequence: when two messages with the same message_id arrived on different partitions within the same hour (which happens during Kafka rebalances when a consumer group is scaled), the second message was incorrectly deduplicated, even though it represented a genuinely new event that happened to be assigned the same message_id due to the ID generation logic using a partition-scoped counter rather than a globally unique identifier. The impact was concentrated on the three largest enterprise customers because their high message volumes made them more likely to experience Kafka rebalances during the pipeline's twice-daily scaling events (scale up at 08:00 Vietnam time, scale down at 22:00 Vietnam time). During each rebalance, approximately 0.3% of messages were re-assigned to different partitions and subsequently dropped by the faulty deduplication logic. Over the course of December, this accumulated to a 15% data loss for the most affected customers. The fix was straightforward: revert the Redis key format to include the partition identifier. However, the remediation required replaying the affected data from Kafka (which retains messages for 7 days) for the December 24-31 period, and requesting the three affected customers to provide their usage logs for December 1-23 (which had already aged out of Kafka retention). The Finance team issued credit notes to the three customers and applied manual billing adjustments for the December period.